

Python - Function Annotations

Function Annotations

The **function annotation** feature of **Python** enables you to add additional explanatory metadata about the arguments declared in a function definition, and also the return data type. They are not considered by **Python interpreter** while executing the **function**. They are mainly for the Python IDEs for providing a detailed documentation to the programmer.

Although you can use the **docstring** feature of Python for documentation of a function, it may be obsolete if certain changes in the function's prototype are made. Hence, the annotation feature was introduced in Python as a result of PEP 3107.

Annotations are any valid Python expressions added to the arguments or return data type. Simplest example of annotation is to prescribe the data type of the arguments. Annotation is mentioned as an expression after putting a colon in front of the argument.

Example

Remember that Python is a dynamically typed language, and doesn't enforce any type checking at runtime. Hence annotating the arguments with **data types** doesn't have any effect while calling the function. Even if non-integer arguments are given, Python doesn't detect any error.

```
def myfunction(a: int, b: int):  
    c = a+b  
    return c
```

```
print (myfunction(10,20))
print (myfunction("Hello ", "Python"))
```

It will produce the following **output** –

```
30
Hello Python
```

Function Annotations With Return Type

Annotations are ignored at runtime, but are helpful for the IDEs and static type checker libraries such as mypy.

You can give annotation for the return data type as well. After the parentheses and before the colon symbol, put an arrow (->) followed by the annotation.

Example

In this example, we are providing annotation for return type.



```
def myfunction(a: int, b: int) -> int:
    c = a+b
    return c
print(myfunction(56,88))
print(myfunction.__annotations__)
```

This will generate the following output –

```
144
{'a': <class 'int'>, 'b': <class 'int'>, 'return': <class 'int'>}
```

Function Annotations With Expression

As using the data type as annotation is ignored at runtime, you can put any expression which acts as the metadata for the arguments. Hence, function may have any arbitrary expression as annotation.

Example

In the below example, we are using expression as a function annotation.

```
def total(x : 'marks in Physics', y: 'marks in chemistry'):
    return x+y
print(total(86, 88))
print(total.__annotations__)
```

Following is the output –

```
174
{'x': 'marks in Physics', 'y': 'marks in chemistry'}
```

Function Annotations With Default Arguments

If you want to specify a **default argument** along with the annotation, you need to put it after the annotation expression. Default arguments must come after the required arguments in the argument list.

Example 1

The following example demonstrates how to provide annotation for default arguments of a function.

```
def myfunction(a: "physics", b:"Maths" = 20) -> int:
    c = a+b
    return c
print (myfunction(10))
```

The function in Python is also an **object**, and one of its attributes is `__annotations__`. You can check with `dir()` function.

```
print (dir(myfunction))
```

This will print the list of myfunction object containing `__annotations__` as one of the attributes.

```
['__annotations__', '__builtins__', '__call__', '__class__', '__closure__', '__co
```

Example 2

The `__annotations__` attribute itself is a **dictionary** in which arguments are keys and annotations their values.

```
def myfunction(a: "physics", b: "Maths" = 20) -> int:
    c = a+b
    return c
print (myfunction.__annotations__)
```

It will produce the following **output** –

```
{'a': 'physics', 'b': 'Maths', 'return': <class 'int'>}
```

Example 3

You may have arbitrary positional and/or arbitrary keyword arguments for a function. Annotations can be given for them also.

```
def myfunction(*args: "arbitrary args", **kwargs: "arbitrary keyword args") ->
int:
    pass
print (myfunction.__annotations__)
```

It will produce the following **output** –

```
{'args': 'arbitrary args', 'kwargs': 'arbitrary keyword args', 'return': <class
```

Example 4

In case you need to provide more than one annotation expressions to a function argument, give it in the form of a dictionary object in front of the argument itself.

```
def division(num: dict(type=float, msg='numerator'), den: dict(type=float,
msg='denominator')) -> float:
    return num/den
print (division.__annotations__)
```

It will produce the following **output** –

```
{'num': {'type': <class 'float'>, 'msg': 'numerator'}, 'den': {'type': <class 'f
```

TOP TUTORIALS

- Python Tutorial
- Java Tutorial
- C++ Tutorial
- C Programming Tutorial
- C# Tutorial
- PHP Tutorial
- R Tutorial
- HTML Tutorial



Chapters ▾

Categories ≡

SQL Tutorial

TRENDING TECHNOLOGIES

- Cloud Computing Tutorial
- Amazon Web Services Tutorial
- Microsoft Azure Tutorial
- Git Tutorial
- Ethical Hacking Tutorial

[Docker Tutorial](#)
[Kubernetes Tutorial](#)
[DSA Tutorial](#)
[Spring Boot Tutorial](#)
[SDLC Tutorial](#)
[Unix Tutorial](#)

CERTIFICATIONS

[Business Analytics Certification](#)
[Java & Spring Boot Advanced Certification](#)
[Data Science Advanced Certification](#)
[Cloud Computing And DevOps](#)
[Advanced Certification In Business Analytics](#)
[Artificial Intelligence And Machine Learning](#)
[DevOps Certification](#)
[Game Development Certification](#)
[Front-End Developer Certification](#)
[AWS Certification Training](#)
[Python Programming Certification](#)

COMPILERS & EDITORS

[Online Java Compiler](#)
[Online Python Compiler](#)
[Online Go Compiler](#)
[Online C Compiler](#)
[Online C++ Compiler](#)
[Online C# Compiler](#)
[Online PHP Compiler](#)
[Online MATLAB Compiler](#)
[Online Bash Terminal](#)
[Online SQL Compiler](#)
[Online Html Editor](#)

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.